

# Product Categorization at Scale : From Unsupervised to Supervised

Synthèse d'avancement – Architecture Globale et Résultats

Salah Eddine NEJJARI | Tao DANG TRAN | Thomas FAVRE | Yohan ANDRIAMAMPIONONA

Projet Stat'App – ENSAE Paris | 2025–2026

**Encadrement :** Guillaume Lochon, Ilan Benchetrit (Criteo) — Nicolas Chopin (ENSAE)

**Résumé.** Ce projet vise à projeter automatiquement un catalogue massif de produits vers la Google Product Taxonomy. Dans un but d'exploration, **la première phase du projet a été volontairement menée sans données annotées (Ground Truth)**, via une approche d'optimisation de flux réseau (Min-Cost Max-Flow). L'accès ultérieur au Ground Truth a permis de basculer vers un **Pipeline Hiérarchique à Double Modèle (Dual Embedding Pipeline)** supervisé. Ce rapport détaille ces deux phases, leurs fondements mathématiques, les défis d'ingénierie matérielle, et l'impact métier massif de cette transition.

## 1 Phase 1 : Exploration Non-Supervisée (Sans Ground Truth)

Au démarrage du projet, l'absence volontaire de Ground Truth nous a poussés à concevoir une méthode d'assignation globale non-supervisée pour le Level 1, en exploitant une unique information externe : **le quota (volume exact) de produits attendu par catégorie racine**.

### 1.1 Représentation vectorielle et agrégation hiérarchique

Chaque produit est représenté par la concaténation de ses métadonnées (**title, description, brand**), nettoyées et filtrées pour ne conserver que la langue anglaise. Les textes sont projetés dans un espace vectoriel dense via le modèle **all-MiniLM-L6-v2** (SentenceTransformers) pour obtenir un vecteur produit  $\mathbf{v}_i$ .

Pour représenter les catégories cibles, nous avons exploité la structure de la taxonomie. L'embedding d'une catégorie Level 1  $C_j$  n'est pas seulement le vecteur de son nom, mais **le barycentre de sa branche complète**. Soit  $\mathcal{D}(C_j)$  l'ensemble contenant  $C_j$  et tous ses descendants dans le graphe **NetworkX**; le prototype de la catégorie est défini par :

$$\mathbf{z}_j = \frac{1}{|\mathcal{D}(C_j)|} \sum_{c \in \mathcal{D}(C_j)} \text{Encoder}(c)$$

La matrice de similarité entre les  $N$  produits et les  $M$  catégories racines est calculée via la similarité cosinus :  $S_{i,j} = \cos(\mathbf{v}_i, \mathbf{z}_j)$ .

### 1.2 Assignation globale : Algorithme Min-Cost Max-Flow

Une assignation naïve (*argmax* sur la similarité) aurait violé les quotas connus de notre catalogue. Pour forcer le respect de la distribution macroscopique, nous avons modélisé le problème comme un réseau de transport biparti résolu par **OR-Tools**.

Le graphe est construit avec une Source  $\mathcal{S}$ , un Puits  $\mathcal{T}$ ,  $N$  nœuds produits et  $M$  nœuds catégories :

- **Arcs Source  $\rightarrow$  Produits** : Capacité = 1, Coût = 0.
- **Arcs Produits  $\rightarrow$  Catégories** : Pour limiter l'explosion combinatoire, seuls les  $K$  meilleurs candidats ( $K = 10$ ) par produit sont conservés. Capacité = 1. Le coût est défini pour pénaliser les faibles similarités : Coût $_{i,j} = \lfloor (1 - S_{i,j}) \times 10^5 \rfloor$ .
- **Arcs Catégories  $\rightarrow$  Puits** : Capacité = Quota de la catégorie (ex : 20 000 pour *furniture*), Coût = 0.

L'algorithme *Min-Cost Max-Flow* trouve l'assignation globale exacte minimisant la "distance sémantique" totale tout en respectant strictement les volumes par catégorie.

## 2 Phase 2 : Architecture Supervisée et Deep Learning

Si la Phase 1 garantissait le respect des quotas, elle restait aveugle aux frontières sémantiques réelles de notre catalogue. **L'obtention du Ground Truth** a marqué un tournant décisif. Elle nous a permis d'abandonner le forçage global (Min-Cost Max-Flow) pour concevoir un **Pipeline Hiérarchique à Deux Étapes (Two-Stage)** supervisé, séquentiel et beaucoup plus précis.

### 2.1 Étape 1 : Verrouillage de la racine (Level 1) - Séquence Supervisée

Pour prédire le premier niveau de la taxonomie de manière extrêmement fiable, notre architecture suit désormais une séquence chronologique stricte en trois temps :

1. **Fine-Tuning du modèle d'Embedding** : L'espace vectoriel "généraliste" initial est d'abord ré-entraîné (*fine-tuné*) spécifiquement sur nos données annotées (Ground Truth). Le modèle apprend à modifier ses poids pour rapprocher artificiellement les produits d'une même catégorie racine et éloigner ceux de catégories différentes.
2. **Génération des vecteurs (Embedding)** : Une fois ce modèle ajusté à notre métier, il agit comme un extracteur de caractéristiques. Le texte de chaque produit du catalogue y est injecté pour être transformé en un vecteur dense optimisé pour notre taxonomie.
3. **Classification finale par XGBoost** : Ces vecteurs (et non plus les textes bruts) sont ensuite donnés en entrée à un algorithme supervisé (XGBoost). Entraîné sur le Ground Truth, XGBoost apprend les frontières de décision exactes dans ce nouvel espace vectoriel, garantissant une prédiction robuste sans avoir à forcer arbitrairement des quotas.

## 2.2 Étape 2 : Raffinement topologique et sémantique (Level 2+) - Zero-Shot

Une fois la catégorie Level 1 figée par XGBoost, le système interroge le graphe **NetworkX** de la taxonomie pour lister *uniquement* les enfants valides de cette racine.

Pour départager ces sous-catégories, nous n'utilisons pas le modèle fine-tuné précédemment (car il a été mathématiquement biaisé pour séparer les Level 1), mais un **second modèle d'embedding** expert en Similarité Sémantique (STS / Zero-Shot). Ce dernier compare le vecteur du texte du produit avec les vecteurs de descriptions de sous-catégories enrichies au préalable par un LLM. La sous-catégorie présentant la plus forte similarité cosinus est alors retenue.

## 3 Modélisation Mathématique de la Phase 2 (Level 1)

### 3.1 Fine-Tuning : Batch Hard Triplet Loss et Contraintes matérielles

Pour adapter l'espace vectoriel  $f_\theta$  à notre taxonomie, nous avons appliqué une **Batch Hard Triplet Loss**. L'algorithme rapproche chaque produit d'ancrage de son exemple positif le plus éloigné et l'écarte de son exemple négatif le plus proche :

$$\mathcal{L}_{\text{BHTL}}(\theta) = \frac{1}{B} \sum_{i=1}^B \max \left( 0, m + \max_{x_p \in \mathcal{P}(i)} D(f_\theta(x_a^{(i)}), f_\theta(x_p)) - \min_{x_n \in \mathcal{N}(i)} D(f_\theta(x_a^{(i)}), f_\theta(x_n)) \right)$$

Nos machines étant équipées de puces Apple (MPS) avec une mémoire unifiée limitée, l'entraînement a nécessité plusieurs heuristiques d'ingénierie :

- **Sous-échantillonnage** : Entraînement restreint à un échantillon stratifié de 30 000 produits.
- **Micro-Batching** : Taille de batch drastiquement réduite à  $B = 4$  ou  $8$  pour éviter la saturation VRAM.
- **Troncature** : Longueur maximale des textes plafonnée à 128 tokens ( $\mathcal{O}(N^2)$  pour l'attention).

### 3.2 Classification finale : Modèle XGBoost

Une fois les représentations apprises par le modèle fine-tuné, les vecteurs produits  $\mathbf{v}_i \in \mathbb{R}^{384}$  sont injectés dans un classifieur **XGBoost (eXtreme Gradient Boosting)**. Contrairement à des méthodes paramétriques classiques, XGBoost construit un ensemble additif de  $K$  arbres de régression  $\mathcal{F}$  pour prédire la catégorie :

$$\hat{y}_i^{(K)} = \sum_{k=1}^K f_k(\mathbf{v}_i), \quad f_k \in \mathcal{F}$$

L'apprentissage se fait de manière itérative. À l'itération  $t$ , la fonction d'objectif à minimiser comprend une perte empirique  $l$  (ex : log-loss multiclasse) et un terme de régularisation  $\Omega$  contrôlant la complexité de l'arbre  $f_t$  :

$$\mathcal{O}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{v}_i)) + \Omega(f_t)$$

Pour optimiser rapidement cet objectif sur nos vecteurs de dimension 384, XGBoost utilise une approximation de Taylor à l'ordre 2 :

$$\mathcal{O}^{(t)} \approx \sum_{i=1}^n \left[ l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{v}_i) + \frac{1}{2} h_i f_t^2(\mathbf{v}_i) \right] + \Omega(f_t)$$

avec les gradients  $g_i = \partial_{\hat{y}_{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$  et hessiens  $h_i = \partial_{\hat{y}_{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$ . La régularisation  $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|w\|_2^2$  pénalise le nombre de feuilles  $T$  et la norme des scores des feuilles  $w$ . Cette modélisation s'est avérée redoutable pour capturer les frontières non-linéaires résiduelles dans l'espace d'embedding.

## 4 Impact Métier : Analyse de l'Accuracy "Avant / Après"

Le passage de la méthode non-supervisée (Phase 1, contrainte par les quotas) au modèle Deep Learning supervisé (Phase 2) a généré un saut de performance spectaculaire, validé sur un hold-out test de 2000 produits annotés. **L'Accuracy globale atteint 91%.**

| Catégorie (Level 1)       | Phase 1 (Min-Cost Max-Flow)          | Phase 2 (XGBoost sur GT) | Progression         |
|---------------------------|--------------------------------------|--------------------------|---------------------|
| Apparel & accessories     | 76.7 %                               | <b>97.0 %</b>            | (+ 20.3 pts)        |
| Toys & games              | 61.8 %                               | <b>90.0 %</b>            | (+ 28.2 pts)        |
| Software                  | 30.9 %                               | <b>84.0 %</b>            | (+ 53.1 pts)        |
| Food, beverages & tobacco | 22.7 %                               | <b>69.0 %</b>            | (+ 46.3 pts)        |
| Arts & entertainment      | 13.3 %                               | <b>68.0 %</b>            | (+ 54.7 pts)        |
| <b>Précision Globale</b>  | $\approx 65.0 \%$ (Moyenne pondérée) | <b>91.0 %</b>            | <b>(+ 26.0 pts)</b> |

TABLE 1 – Comparatif Avant/Après : L'apprentissage supervisé permet au modèle de comprendre les frontières sémantiques complexes plutôt que de "forcer" des produits dans des catégories pour remplir des quotas.

## 5 Prédiction Level 2 : Graphe, Zero-Shot et Active Learning

### 5.1 Modélisation de la Taxonomie via Graphe Orienté

Le Level 1 étant verrouillé à 91% d'Accuracy, l'enjeu du *Level 2* est de garantir une cohérence hiérarchique absolue. La Google Product Taxonomy est mathématiquement modélisée sous forme de graphe dirigé  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  via **NetworkX**. L'espace de recherche pour un produit  $i$  est ainsi strictement restreint aux nœuds enfants directs de la prédiction Level 1 :

$$\mathcal{C}_{\text{candidates}}(i) = \{v \in \mathcal{V} \mid (\hat{y}_i^{(1)}, v) \in \mathcal{E}\}$$

### 5.2 Enrichissement Sémantique par LLM et Retrieval STS

Un défi majeur du Zero-Shot Retrieval réside dans la pauvreté sémantique des labels bruts de la taxonomie (ex : les mots "Handbags" ou "Diapering" manquent de contexte pour un modèle mathématique). Pour y pallier, nous avons **augmenté l'expressivité sémantique** de nos ancres cibles en utilisant un Large Language Model (LLM). Chaque catégorie Level 2 s'est vue attribuer une description contextuelle dense générée automatiquement (ex : pour *kitchen & dining*, l'ancre devient *"Cookware, utensils, and appliances used for preparing, serving, and consuming food"*).

Le premier modèle d'embedding ayant été fortement biaisé par le Fine-Tuning pour discriminer les Level 1, nous introduisons ici un second modèle (Dual Embedding), spécialisé en *Semantic Textual Similarity* (STS) et *Zero-Shot* (BAAI/bge-small-en-v1.5). L'ancre enrichie est projetée en un vecteur de référence  $Z_d^{(2)}$ . Le texte du produit subit la même projection  $v_i$ . La prédiction finale maximise la similarité cosinus sur le sous-graphe autorisé :

$$\hat{y}_i^{(2)} = \arg \max_{d \in \mathcal{C}_{\text{candidates}}(i)} \frac{v_i^\top Z_d^{(2)}}{\|v_i\|_2 \|Z_d^{(2)}\|_2}$$

Concrètement, la **classification finale s'opère par une recherche du plus proche voisin (1-NN) contrainte spatialement**. Une fois le score de similarité cosinus calculé entre le produit et toutes les descriptions candidates autorisées par le graphe **NetworkX**, l'algorithme sélectionne simplement la sous-catégorie dont le vecteur d'ancre forme l'angle le plus fermé (score le plus proche de 1) avec le produit. C'est cette mécanique purement géométrique qui permet au système de classer avec précision des millions de produits, sans nécessiter de réentraînement supervisé (Target Labels) pour les niveaux inférieurs, réalisant ainsi une véritable *Zero-Shot Classification*.

## 6 Perspectives et Conclusion

L'accès au Ground Truth a transformé notre approche d'assignation globale non-supervisée en un véritable système de Deep Learning rigoureux. L'apprentissage des représentations via Triplet Loss couplé à XGBoost excelle (91% d'Accuracy au Level 1). L'analyse "Avant/Après" prouve la supériorité de l'apprentissage supervisé sur un espace vectoriel spécialisé, par rapport à notre approche initiale d'assignation sous contrainte de quotas (Min-Cost Max-Flow sur embeddings génériques). Cette évolution a permis des gains dépassant les +50 points sur les catégories sémantiquement ambiguës. L'hybridation de l'architecture finale (racine verrouillée par supervision, puis raffinement topologique via Graphe et LLM+Zero-Shot) pose des fondations industrielles solides.

La suite du projet s'articulera autour de deux axes majeurs d'optimisation mathématique et algorithmique pour l'exploration profonde du catalogue (Levels 3, 4+) :

1. **Exploration des métriques de distance spatiale** : Actuellement basé sur la similarité cosinus (qui mesure l'angle entre deux vecteurs), le Retrieval STS pourrait être testé avec la distance Euclidienne ( $L_2$ ) ou le produit scalaire (*Dot Product*). Selon la topologie exacte de l'espace latent post-finetuning, une métrique Euclidienne pourrait mieux capturer les variations de magnitude induites par l'hétérogénéité de la longueur des descriptions produits.
2. **Prévention de la propagation d'erreur : Soft Decoding et Beam Search** : Actuellement, notre pipeline repose sur un décodage strict (*Hard/Greedy Decoding*) : à chaque profondeur, seul le meilleur nœud local est

conservé. Le danger mathématique de cette approche est la **propagation d'erreur en cascade** : si la bonne racine au Level 1 obtient une probabilité de 48% face à une mauvaise à 49%, la bonne branche est définitivement élaguée. Pour y remédier, nous généraliserons l'exploration du graphe :

**Pour le Level 2 (Soft Hierarchical Decoding)** : Au lieu de restreindre la recherche au top-1 du Level 1, nous autoriserons la recherche sur les enfants des  $B$  meilleurs parents (ex :  $B = 3$ ). La décision finale  $\hat{y}^{(2)}$  maximisera une fonction d'objectif hybride pondérant la probabilité supervisée du parent  $\pi(d)$  et la similarité cosinus de l'enfant  $d$ , avec un hyperparamètre de compromis  $\alpha$  :

$$\hat{y}^{(2)} = \arg \max_{d \in \bigcup_{b=1}^B \mathcal{C}_b} \left[ \alpha \cdot \underbrace{\mathbb{P}(\hat{y}^{(1)} = \pi(d) \mid \mathbf{v}_i)}_{\text{Confiance Supervisée L1}} + (1 - \alpha) \cdot \underbrace{S_{\cos}(d)}_{\text{Similarité Sémantique L2}} \right]$$

**Généralisation aux Niveaux Profonds (Levels 3, 4+) via Beam Search** : Grâce à **NetworkX**, cette logique de "filet de sécurité" est virtuellement scalable à une profondeur infinie via un **algorithme de Beam Search**. Le Beam Search maintient en permanence les  $B$  meilleures trajectoires hiérarchiques en mémoire. Soit  $p = (y^{(1)}, \dots, y^{(L)})$  un chemin de profondeur  $L$ . On définit son score cumulé comme :

$$S_L(p) = \sum_{l=1}^L \gamma_l \text{Score}(y^{(l)} \mid x_i)$$

Lors de la descente au niveau  $L + 1$ , l'algorithme génère l'ensemble des expansions possibles à partir du faisceau courant  $\mathcal{H}_L$  :

$$\mathcal{E}_{L+1} = \bigcup_{p \in \mathcal{H}_L} \left\{ p \oplus c \mid c \in \text{Successors}_{\mathcal{G}}(y^{(L)}) \right\}$$

L'algorithme trie  $\mathcal{E}_{L+1}$  et ne conserve que les  $B$  trajectoires maximisant ce score global. Cette optimisation globale permettra au modèle de compenser une incertitude locale (ex : au L1) par une certitude absolue acquise dans les niveaux inférieurs (L3, L4), évitant ainsi l'élagage prématuré des bonnes branches sémantiques.